

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO4 ASSESSMENT TOOL 1 – Test Case Design Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 1 – Test Case Design Assignment
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	Rashmi Naurene	Reg. No.:	192521357
Date:	15.08.26	Duration:	60 minutes

Code of Conduct

I _____ Rashmi Naurene (192521357) _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Rashmi _____

Annexure A – Test Case Design Assignment

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design and document comprehensive test cases** to verify the correctness, functionality, reliability, and usability of the proposed system.

Your test case design should include:

1. Identify the **functional and non-functional requirements** to be tested.
2. Identify the **test scenarios** for the assigned problem statement.
3. Prepare detailed **test cases** covering normal, boundary, invalid, and exceptional conditions.
4. Include **Test Case ID, Test Scenario, Test Steps, Test Data, Expected Result, Actual Result, and Pass/Fail Status**.
5. Apply appropriate **test design techniques** such as Equivalence Partitioning, Boundary Value Analysis, Decision Table, or State Transition Testing wherever applicable.
6. Ensure that the test cases provide adequate **requirement and functional coverage** for the assigned system.

Deliverable: Submit a Test Case Design document containing the identified scenarios, test cases, test data, expected results, and test coverage based on the individual problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Test Scenario Identification(15 %)	Clearly identifies all relevant functional/non-functional requirements and comprehensive test scenarios	Identifies most requirements and scenarios	Identifies basic requirements and scenarios	Requirements/scenarios are incomplete or unclear
Test Case Design & Completeness(25 %)	Comprehensive test cases covering normal, boundary, invalid, and exceptional conditions	Covers most important conditions with minor gaps	Covers basic conditions but misses several important cases	Test cases are very limited or incomplete
Test Data & Expected Results(20%)	Appropriate test data with precise, measurable expected results for every test case	Mostly appropriate test data and expected results	Some test data/results are unclear or incomplete	Test data and expected results are largely missing/incorrect
Application of Test Design Techniques(15%)	Correctly applies multiple techniques such as Equivalence Partitioning, Boundary Value Analysis, Decision Table, and State Transition	Correctly applies at least two relevant techniques	Applies one technique with limited justification	Techniques are not applied or incorrectly applied
Traceability & Test Coverage(15%)	Excellent mapping between requirements, scenarios, and test cases with strong coverage	Good traceability and coverage with minor gaps	Partial traceability and moderate coverage	Poor/no traceability and inadequate coverage
Documentation & Presentation(10 %)	Well-structured, clear, professional, consistent, and easy to understand	Well-organized with minor presentation issues	Adequately organized but has some clarity/formatting issues	Poorly organized, unclear, or incomplete documentation

Note: This test case design targets the actual, confirmed functionality of the forked repository under test — the Employee Performance Management System (Java/Maven/JavaFX, PostgreSQL-backed) — since this is the real, running system continued from the earlier Git & Docker and Code Review assignments. Test cases are written against the documented functional requirements (Add/Edit/Delete/Restore Employee, Evaluate Employee, Grades, Project management), not against features that are not yet implemented. “Actual Result” and “Pass/Fail” columns are left blank for you to fill in once you execute each test — see Section 7 for exactly how to do that.

1 · Requirements Under Test

1.1 Functional Requirements

ID	Functional Requirement
FR-01	Add a new employee record
FR-02	Edit an existing employee record
FR-03	Delete an employee (move to Trash)
FR-04	Restore an employee from Trash
FR-05	Evaluate an employee (performance, attendance, soft-skill, hard-skill)
FR-06	View all employees
FR-07	View Trash
FR-08	Assign / view employee grade
FR-09	Grade existence check (prevent duplicate grading)
FR-10	Clear input fields
FR-11	View, add, edit, and delete a project

1.2 Non-Functional Requirements

Category	Non-Functional Requirement
Performance	CRUD operations on employee/project records should complete within an acceptable, noticeably-responsive time on typical local hardware.
Reliability	Employee data must persist correctly across application restarts and container restarts (PostgreSQL data volume).
Data Integrity	The multi-step employee-to-trash operation should not leave the database in an inconsistent state if one step fails.
Usability	Invalid or incomplete input must be rejected with a clear alert rather than silently failing or crashing.
Portability	The application must behave the same way whether run directly via Maven or inside the Docker container.
Security	Database access should not expose credentials to the end user or fail insecurely.

NFR-03 (data integrity) and Security directly follow up on the transaction-management and hard-coded-credential findings raised in the Code Review & Repository Evaluation report — this test plan is written to actually verify those risk areas, not just re-describe them.

2 · Test Scenarios

Scenario ID	Scenario Description	Related FR(s)
TS-01	Add a new employee with complete, valid data	FR-01
TS-02	Attempt to add an employee with missing/invalid data	FR-01
TS-03	Edit an existing employee's details	FR-02
TS-04	Delete an employee (move to Trash)	FR-03
TS-05	Restore a previously deleted employee from Trash	FR-04
TS-06	Evaluate an employee's performance/attendance/skills	FR-05
TS-07	View the full employee list	FR-06
TS-08	View the Trash list	FR-07
TS-09	Assign a grade to an employee	FR-08
TS-10	Attempt to assign a duplicate grade to the same employee	FR-09
TS-11	Clear all input fields on the form	FR-10
TS-12	Add, edit, and delete a project	FR-11
TS-13	Verify data persists after restarting the Docker containers	NFR-Reliability
TS-14	Verify application behaviour when the database is unreachable	NFR-Reliability

3 · Test Design Techniques Applied

3.1 Equivalence Partitioning (EP)

Input fields are grouped into valid and invalid classes, and only one representative value from each class is tested, rather than testing every possible value. For example, the employee name field partitions into: a non-empty valid string (valid class) versus an empty string or null (invalid class); the department field similarly partitions into an existing, valid department versus an unrecognized one.

3.2 Boundary Value Analysis (BVA)

Errors cluster at the edges of valid ranges, so values exactly at, just below, and just above a boundary are tested explicitly. This applies directly to any bounded numeric field in the evaluation module — for example, if soft-skill/hard-skill/attendance scores are recorded on a 0–100 scale, the boundary values tested are -1, 0, 1, 99, 100, and 101.

3.3 Decision Table Testing

The employee evaluation logic combines multiple independent inputs (attendance, soft-skill, hard-skill) to arrive at a grade outcome. A decision table enumerates the combinations of high/low values for each input against the resulting grade, ensuring the evaluation logic is tested for combinations of conditions, not just each field in isolation.

Rule	Attendance	Soft-Skill	Hard-Skill	Expected Grade Outcome
R1	High	High	High	Highest grade band
R2	High	High	Low	Mid grade band

R3	Low	High	High	Mid grade band (attendance penalty)
R4	Low	Low	Low	Lowest grade band

3.4 State Transition Testing

An employee record moves through a defined lifecycle: Active → Trashed (via Delete) → either Restored (back to Active) or permanently removed. State transition testing verifies each valid transition behaves correctly and that no operation is available from a state where it should not be — for example, an already-Trashed employee should not be editable as if Active until it is restored.

4 · Detailed Test Cases

TC ID	Scenario	Test Case Title	Test Steps	Test Data	Expected Result
TC-01	TS-01	Add employee — valid data (Normal, EP-valid)	1. Open Add Employee form. 2. Enter name, department, hire date. 3. Click Save.	Name: "Anita Rao", Dept: "Engineering", HireDate: 2024-01-15	Employee is saved and appears in the employee list.
TC-02	TS-02	Add employee — empty name (Invalid, EP-invalid)	1. Open Add Employee form. 2. Leave name blank. 3. Click Save.	Name: "" (empty)	Save is rejected; a validation alert is shown; no record is created.
TC-03	TS-02	Add employee — invalid hire date format (Invalid)	1. Open Add Employee form. 2. Enter a malformed date. 3. Click Save.	HireDate: "31-31-2024"	Save is rejected; a validation alert is shown.
TC-04	TS-03	Edit employee — update department (Normal)	1. Select an existing employee. 2. Change department field. 3. Click Save/Update.	New Dept: "HR"	Record is updated; new department is reflected in the employee list.
TC-05	TS-03	Edit employee — clear required field then save (Invalid)	1. Select an existing employee. 2. Clear the name field. 3. Click Save/Update.	Name: "" (cleared)	Update is rejected with a validation alert; original data is unchanged.
TC-06	TS-04	Delete employee — move to Trash (Normal)	1. Select an existing employee. 2. Click Delete/Move to Trash.	Any existing employee record	Employee no longer appears in the main list and appears in Trash.
TC-07	TS-04	Delete employee — confirm grade is also handled (State/Integrity)	1. Select an employee who has an assigned grade. 2. Delete the employee. 3. Inspect the database/grade view.	Employee with an existing grade record	Employee and associated grade are moved/removed consistently — no orphaned grade record remains.
TC-08	TS-05	Restore employee from Trash (Normal, State Transition)	1. Open Trash view. 2. Select a trashed employee. 3. Click Restore.	Previously trashed employee	Employee reappears in the main employee list; is removed from Trash.
TC-09	TS-05	Restore employee — Trash is empty (Exceptional)	1. Ensure Trash is empty. 2. Open Trash view.	No trashed employees	Trash view displays empty state; no error/crash occurs.
TC-10	TS-06	Evaluate employee — all scores mid-range (Normal, Decision Table R2/R3)	1. Select an employee. 2. Enter attendance, soft-skill, hard-skill scores. 3. Submit evaluation.	Attendance: 80, Soft-skill: 85, Hard-skill: 60	Grade is computed and displayed per the evaluation logic (mid grade band).

TC-11	TS-06	Evaluate employee — score at lower boundary (BVA)	1. Select an employee. 2. Enter a score of exactly 0. 3. Submit evaluation.	Hard-skill score: 0	Score of 0 is accepted as valid input; grade reflects the lowest band.
TC-12	TS-06	Evaluate employee — score at upper boundary (BVA)	1. Select an employee. 2. Enter a score of exactly 100. 3. Submit evaluation.	Hard-skill score: 100	Score of 100 is accepted as valid input; grade reflects the highest band.
TC-13	TS-06	Evaluate employee — score just below valid range (BVA, Invalid)	1. Select an employee. 2. Enter a score of -1. 3. Submit evaluation.	Attendance score: -1	Input is rejected with a validation alert.
TC-14	TS-06	Evaluate employee — score just above valid range (BVA, Invalid)	1. Select an employee. 2. Enter a score of 101. 3. Submit evaluation.	Attendance score: 101	Input is rejected with a validation alert.
TC-15	TS-07	View all employees — list is populated (Normal)	1. Open the main employee view.	Two or more existing employees	All active (non-trashed) employees are listed correctly.
TC-16	TS-07	View all employees — no employees exist (Exceptional)	1. Ensure no employees exist. 2. Open the main employee view.	Empty employee table	An empty list/appropriate empty-state is shown; no crash occurs.
TC-17	TS-09	Assign grade to employee — first time (Normal)	1. Select an employee with no existing grade. 2. Enter a grade. 3. Save.	Grade: "A"	Grade is saved and displayed against the employee.
TC-18	TS-10	Assign grade — duplicate grade check (Invalid, Grade Existence Check)	1. Select an employee who already has a grade. 2. Attempt to assign another grade without updating. 3. Save.	Employee with existing grade "A"; attempted new grade "B"	System detects the existing grade and either blocks the duplicate or routes to an update path, per FR-09.
TC-19	TS-11	Clear input fields (Normal)	1. Fill in several fields on the Add Employee form. 2. Click Clear.	Partially filled form	All fields are reset to empty/default; no data is submitted.
TC-20	TS-12	Add / edit / delete a project (Normal)	1. Add a new project. 2. Edit its details. 3. Delete it.	Project name: "Skill-Gap Rollout Phase 1"	Project is created, updated, and removed correctly at each step.
TC-21	TS-13	Data persists after container restart (Reliability)	1. Add an employee. 2. Run `docker compose down`. 3. Run `docker compose up -d`. 4. View the employee list.	One newly added employee; PostgreSQL named volume in use	The employee added before the restart is still present after the containers restart.
TC-22	TS-14	Application behaviour when database is unreachable (Exceptional/Reliability)	1. Stop the PostgreSQL container only (`docker stop employee-postgres`). 2. Attempt any database-dependent action in the app.	Database container stopped, application container still running	The application shows a clear connection-error message rather than crashing or hanging silently.

```

NAME                IMAGE                COMMAND                SERVICE    CREATED    STATUS    PORTS
employee-postgres   postgres:16         "docker-entrypoint.s..." postgres   5 days ago Up 2 minutes    0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp

PS C:\Windows\system32\Employee-skill-gap-C03-AT1-> .\mvnw.cmd clean test
[INFO] Scanning for projects...
[INFO]
[INFO] -----< org.example:EmployeePerformanceApp >-----
[INFO] Building EmployeePerformanceApp 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- maven-clean-plugin:2.5:clean (default-clean) @ EmployeePerformanceApp ---
[INFO] Deleting C:\Windows\System32\Employee-skill-gap-C03-AT1\target
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ EmployeePerformanceApp ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] Copying 4 resources
[INFO]
[INFO] --- maven-compiler-plugin:3.13.0:compile (default-compile) @ EmployeePerformanceApp ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 10 source files with javac [debug target 21 module-path] to target\classes
[INFO]
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ EmployeePerformanceApp ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] skip non existing resourceDirectory C:\Windows\System32\Employee-skill-gap-C03-AT1\src\test\resources
[INFO]
[INFO] --- maven-compiler-plugin:3.13.0:testCompile (default-testCompile) @ EmployeePerformanceApp ---
[INFO] Recompiling the module because of changed dependency.
[INFO] Compiling 1 source file with javac [debug target 21 module-path] to target\test-classes
[INFO]
[INFO] --- maven-surefire-plugin:2.12.4:test (default-test) @ EmployeePerformanceApp ---
[INFO] Surefire report directory: C:\Windows\System32\Employee-skill-gap-C03-AT1\target\surefire-reports

-----
T E S T S
-----

Results :

Tests run: 0, Failures: 0, Errors: 0, Skipped: 0

[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 5.542 s
[INFO] Finished at: 2026-08-15T13:16:10+05:30

```

5 · Requirement & Test Coverage Traceability Matrix

Requirement	Covered By Scenario(s)	Covered By Test Case(s)
FR-01	TS-01, TS-02	TC-01, TC-02, TC-03
FR-02	TS-03	TC-04, TC-05
FR-03	TS-04	TC-06, TC-07
FR-04	TS-05	TC-08, TC-09
FR-05	TS-06	TC-10, TC-11, TC-12, TC-13, TC-14
FR-06	TS-07	TC-15, TC-16
FR-07	TS-08	TC-08, TC-09 (Trash view exercised)
FR-08	TS-09	TC-17
FR-09	TS-10	TC-18
FR-10	TS-11	TC-19
FR-11	TS-12	TC-20
NFR — Reliability	TS-13, TS-14	TC-21, TC-22
NFR — Data Integrity	TS-04	TC-07

Every functional requirement identified in Section 1.1 is covered by at least one test scenario and at least one detailed test case, satisfying full requirement traceability for this test plan.

6 · Test Environment

Item	Detail
------	--------

Application	Java/Maven/JavaFX Employee Performance Management System (forked repository)
Database	PostgreSQL 16, run via the project's docker-compose.yml (employee-postgres container)
Build/Test tool	Maven Surefire, via `mvn test` / `.\mvnw.cmd test`
Unit test framework	JUnit 5 (already a project dependency, per pom.xml)
JDK used for local execution	Java 21.0.11 — see Section 7 for why this matters

7 · How to Actually Execute These Tests

This section is the practical companion to Sections 1–5 — it explains which tool to use for each category of test case above, so the Actual Result and Pass/Fail columns can be filled in from real output rather than left as a written exercise.

7.1 Logic-Level Test Cases → JUnit 5 (already in your project)

Your pom.xml already includes JUnit 5, and EmployeeTest.java already demonstrates the pattern. Test cases that check pure logic — validation rules, grade computation, boundary values (TC-02, TC-03, TC-05, TC-10 through TC-14, TC-18) — belong in JUnit test classes, not manual clicking. Add a new test class, for example EmployeeEvaluationTest.java, next to EmployeeTest.java, with one @Test method per test case:

```
@Test
void gradeAtLowerBoundaryIsAccepted() {           // covers TC-11
    Employee e = new Employee(1, "Test User", "Engineering");
    boolean result = evaluator.evaluate(e, /*hardSkill=*/0, 80, 85);
    assertTrue(result);
}

@Test
void scoreBelowRangeIsRejected() {                // covers TC-13
    Employee e = new Employee(1, "Test User", "Engineering");
    assertThrows(IllegalArgumentException.class,
        () -> evaluator.evaluate(e, 50, -1, 85));
}
```

Run the whole suite with:

```
$ .\mvnw.cmd clean test
```

Maven Surefire prints a PASSED/FAILED line per test method and writes a full report to target/surefire-reports/. That report is exactly your “Actual Result” and “Pass/Fail Status” evidence for every logic-level test case — screenshot the console output or attach the surefire report.


```

package org.example.employeeperformanceapp;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

import java.util.Date;

class EmployeeTest {

    @Test
    void testGettersAndSetters() {
        System.out.println("Starting testGettersAndSetters...");

        Date hireDate = new Date();
        Employee employee = new Employee(1, "John Doe", "Engineering", hireDate);

        assertEquals(1, employee.getId());
        System.out.println("Employee ID is correct");

        assertEquals("John Doe", employee.getName());
        System.out.println("Employee Name is correct");

        assertEquals("Engineering", employee.getDepartment());
        System.out.println("Employee Department is correct");

        assertEquals(hireDate, employee.getHireDate());
        System.out.println("Employee HireDate is correct");

        employee.setId(2);
        employee.setName("Jane Smith");
        employee.setDepartment("Marketing");

        assertEquals(2, employee.getId());
        System.out.println("Employee ID updated correctly");

        assertEquals("Jane Smith", employee.getName());
        System.out.println("Employee Name updated correctly");

        assertEquals("Marketing", employee.getDepartment());
        System.out.println("Employee Department updated correctly");
    }
}

```

7.2 Database-Dependent Test Cases → Testcontainers (uses the Docker skills you already have)

Test cases that must touch the real database (TC-01, TC-04, TC-06 through TC-09, TC-15 through TC-17, TC-19, TC-20) are best run as integration tests against a real PostgreSQL instance rather than a mock — and since you already have Docker set up from the previous assignment, the natural tool is Testcontainers, which spins up a throwaway PostgreSQL container automatically for each test run.

- Add the dependency: `org.testcontainers:postgresql` and `org.testcontainers:junit-jupiter` to `pom.xml`.
- Annotate a test class with `@Testcontainers` and declare a `@Container` static `PostgreSQLContainer` — JUnit starts and stops it automatically around your tests.
- Point your `EmployeeDAO` at the container's JDBC URL (`Testcontainers` exposes it via `postgres.getJdbcUrl()`) instead of the hard-coded `localhost:5432` URL flagged in the Code Review report — this is a good opportunity to fix that finding at the same time.

If you'd rather not add Testcontainers for a 60-minute assignment, the simpler fallback is: run ``docker compose up -d`` to start the real `employee-postgres` container, then run the same DAO-level JUnit tests against it directly. This is less isolated (state carries over between runs) but requires no new dependency.

7.3 UI-Level Test Cases → Manual Execution (fastest, most realistic for this assignment)

Test cases that exercise the JavaFX screens directly — clicking Add/Edit/Delete/Restore, checking that alerts appear, checking that fields clear (TC-02, TC-03, TC-05, TC-16, TC-19) — are most practically executed by simply running the application and clicking through each step yourself, since JavaFX is a desktop GUI and full UI automation (via a tool like TestFX) is more setup than a single assignment usually justifies. Run the application with:

```
$ ./mvnw.cmd javafx:run
```

For each UI test case, follow the exact Test Steps column, observe what actually happens, write that into Actual Result, and mark Pass if it matches Expected Result or Fail if it doesn't. A screenshot of each key screen (the alert dialog

for TC-02, the empty-state view for TC-16, etc.) is exactly the evidence your rubric's “Documentation & Presentation” criterion is asking for.

If you want to explore UI automation anyway: TestFX (org.testfx:testfx-junit5) is the standard tool for automating JavaFX specifically — it lets you write `clickOn("#saveButton")` style JUnit tests. It's a good stretch goal, not a requirement, for this assignment.

7.4 Docker/Reliability Test Cases → Your Existing Compose Setup

TC-21 and TC-22 are direct continuations of the previous Git & Docker assignment and use the exact same `docker-compose.yml`:

```
PS C:\Windows\system32> cd "C:\Windows\system32\Employee-skill-gap-C03-AT1-"
PS C:\Windows\system32\Employee-skill-gap-C03-AT1-> docker compose up -d
[+] up 2/2
✔Container employee-postgres      Started
✔Container employee-skill-gap-app Started
```

```
# TC-21: persistence across restart
$ docker compose up -d
# ... add an employee via the running app ...
$ docker compose down
$ docker compose up -d
# ... reopen the app, check the employee is still listed ...

# TC-22: database unreachable
$ docker stop employee-postgres
# ... attempt any DB action in the running app ...
$ docker start employee-postgres      # restore afterwards
```

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Test Scenario Identification(15%)	Clearly identifies all relevant functional/non-functional requirements and comprehensive test scenarios	Identifies most requirements and scenarios	Identifies basic requirements and scenarios	Requirements/ scenarios are incomplete or unclear
Test Case Design & Completeness(25%)	Comprehensive test cases covering normal, boundary, invalid, and exceptional conditions	Covers most important conditions with minor gaps	Covers basic conditions but misses several important cases	Test cases are very limited or incomplete
Test Data & Expected Results(20%)	Appropriate test data with precise, measurable expected results for every test case	Mostly appropriate test data and expected results	Some test data/results are unclear or incomplete	Test data and expected results are largely missing/incorrect
Application of Test Design Techniques(15%)	Correctly applies multiple techniques such as Equivalence Partitioning, Boundary Value Analysis, Decision Table, and State Transition	Correctly applies at least two relevant techniques	Applies one technique with limited justification	Techniques are not applied or incorrectly applied
Traceability & Test Coverage(15%)	Excellent mapping between requirements, scenarios, and test cases with strong coverage	Good traceability and coverage with minor gaps	Partial traceability and moderate coverage	Poor/no traceability and inadequate coverage
Documentation & Presentation(10%)	Well-structured, clear, professional, consistent, and easy to understand	Well-organized with minor presentation issues	Adequately organized but has some clarity/formatting issues	Poorly organized, unclear, or incomplete documentation

Criteria	Mark
Requirement & Test Scenario Identification (15%)	/5
Test Case Design & Completeness (25%)	/4
Test Data & Expected Results (20%)	/4
Application of Test Design Techniques (15%)	/4
Traceability & Test Coverage (15%)	/4
Documentation & Presentation (10%)	/4
TOTAL	/25